

Documentazione Progetto di Reti Informatiche A.A. 2025/2026

Scelte implementative sulla parte di comunicazione

La comunicazione fra client e server avviene esclusivamente tramite protocollo TCP, in modo da garantire l'integrità dei dati e la consistenza del sistema. Ho valutato l'utilizzo del protocollo UDP per l'implementazione della parte P2P del progetto, ma ho preferito non utilizzarlo per evitare situazioni di deadlock (se viene perso un pacchetto il processo non finirà mai la fase di review). Sarebbe possibile mitigare questo problema con altre implementazioni (inserire un quorum ad esempio) ma questo esula dall'obiettivo del progetto.

Connessioni multiple vengono gestite dal server tramite multiplexing, mentre nel client con un approccio misto. L'applicazione infatti genera un thread separato per gestire la comunicazione con i peer, mentre la comunicazione con il server centrale è riservata al thread principale. Ho scelto di implementare così la struttura per avere un migliore isolamento delle task e per modularizzare meglio la struttura del programma. L'opzione del multithreading puro non mi è sembrata ottimale in quanto l'applicazione è pensata per avere un piccolo numero di connessioni e quindi il vantaggio del parallelismo non supera l'overhead del sistema.

Per scambiare i messaggi viene utilizzato un formato standard che include il tipo del messaggio di 32 bit, più la lunghezza del payload (se presente). Infatti, molti messaggi (esempio: CARD_ACK, PING_USER, ecc...) vengono inviati senza payload. I messaggi sono sempre scambiati in binario (facendo attenzione al network order) perché la comunicazione dei client avviene principalmente per scambiarsi id, porte, numeri in generale. Il text protocol avrebbe aiutato nella gestione delle testo delle card, ma in quel caso un array di char fa lo stesso lavoro con meno overhead.

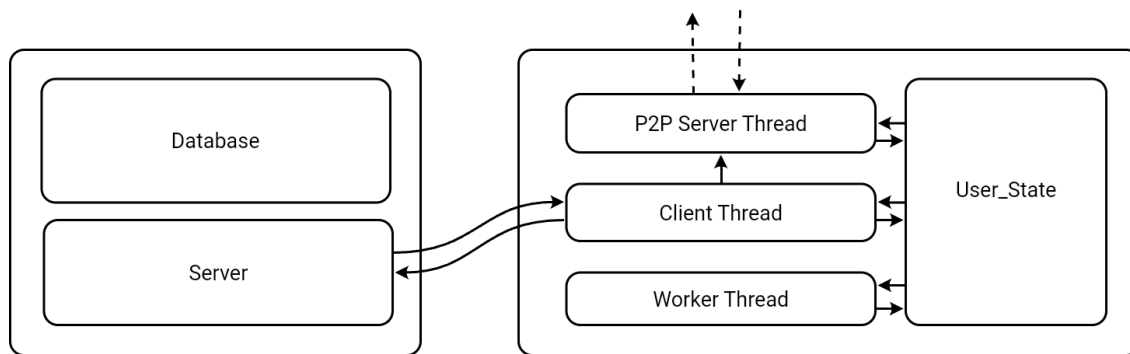
Struttura logica del programma

Ho cercato di implementare il programma il più modularmente possibile. Per fare questo ho definito due "classi" Server e Client che astraggono la comunicazione fra i processi, e la parte "protocol" che gestisce interamente lo scambio dei messaggi. Avendo fatto questo la parte di basso livello è stata gestita esclusivamente tramite una API. Un altro vantaggio di questa astrazione è che la parte server del P2P e il server centrale sono gestiti tramite lo stesso codice, evitando di dover ripetere del codice da una parte all'altra. Si può notare anche che la struttura di Client e Server è molto simile, e infatti si potrebbe pensare di unire i due sotto un'unica "classe". Tuttavia, essendo C non orientato agli oggetti ho preferito non forzare troppo l'astrazione.

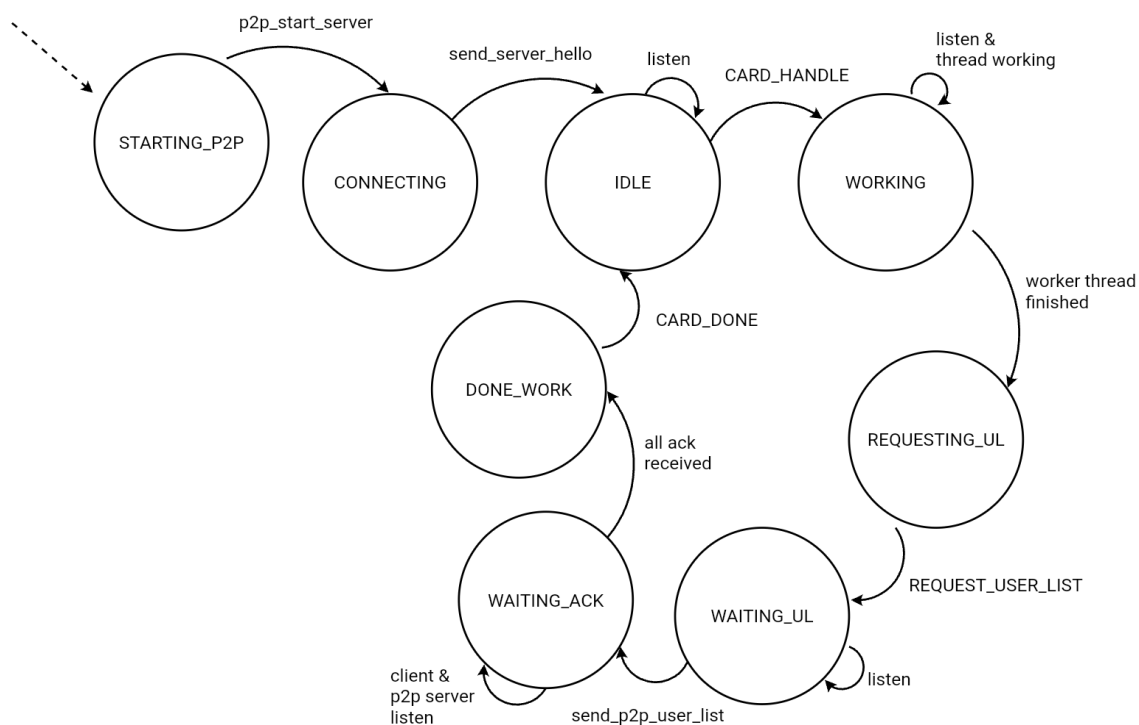
Il client e il server hanno poi due strutture parallele per gestire i dati. Il server dispone del suo Database, che include le strutture per memorizzare gli Utenti e le Card. L'implementazione non è ottima né dal punto di vista degli algoritmi né da quello delle strutture dati, tuttavia trattandosi di un progetto ristretto ho preferito una implementazione semplificata con delle linked list. Si potrebbe pensare in potenza di implementare metodi migliori per ottimizzare l'inserimento e la rimozione degli utenti, ad esempio. Aggiungo anche che l'accesso al database è effettuato esclusivamente tramite funzioni, per garantire la consistenza. Per quanto riguarda l'utente invece, ha uno stato globale condiviso tra vari thread; proprio per questo motivo, l'accesso è ristretto a solo tramite funzioni ed è protetto tramite mutex.

L'ultima struttura implementativa di cui voglio parlare è il thread worker. Questo viene eseguito quando il client riceve una card e simula il suo svolgimento. Il vantaggio che questa implementazione

da rispetto ad una normale sleep è la possibilità di rispondere a messaggi sia dei peer che del server nel mentre eseguiamo il task.



I tre thread del processo Utente comunicano attraverso l'user state, in particolare attraverso la sua macchina a stati. In ogni stato ogni thread esegue un task e finisce di eseguirlo quando si esce dallo stato.



La state machine dell'applicazione utente può essere vista a grandi linee in questo modo. Sono presenti anche altri due stati (STATE_SHUTTING_DOWN e STATE_DISCONNECTING) in cui si entra solo in caso di errore o di terminazione e servono a liberare tutte le risorse prese.

Compilazione

Il programma può essere compilato semplicemente con make (sono già abilitati -Wall -Wextra e -Wpedantic). Viene lanciato con ./utente <porta> e ./lavagna come da specifica. "Make clean" invece rimuove i file oggetto e gli eseguibili.